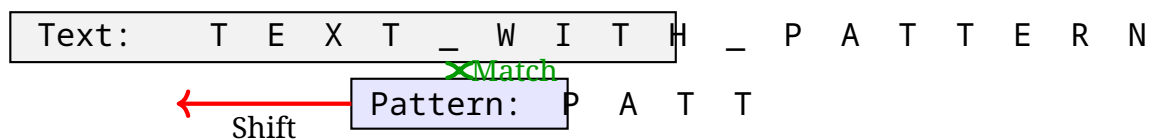

Advanced Data Structures Academic

Student Notes

UNIT - V: Pattern Matching Algorithms



Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management
(UGC Autonomous, R25 Regulation)

Contents

1	Introduction to Pattern Matching	2
2	Naive / Brute Force String Matching	2
2.1	Algorithm Mechanics	2
2.2	Complexity Analysis	2
3	Knuth-Morris-Pratt (KMP) Algorithm	2
3.1	The Prefix Function (Pi Table / Failure Function)	3
3.2	KMP Search Execution	3
3.3	Complexity Analysis	3
4	Boyer-Moore Algorithm	4
4.1	The Bad Character Heuristic	4
4.2	The Good Suffix Heuristic	4
4.3	Shift Selection	4
5	Horspool's Algorithm	4
5.1	The Shift Table Calculation	5
5.2	Complexity Analysis	5
6	Rabin-Karp Algorithm	5
6.1	Rolling Hash Function	5
6.2	Handling Collisions	6
6.3	Complexity Analysis	6
7	Algorithm Comparison Summary	6

1 Introduction to Pattern Matching

The Pattern Matching problem is a fundamental challenge in computer science: given a large string of text T of length n , and a pattern string P of length m (where $m \leq n$), find one or all occurrences of P inside T .

Definition 1.1 (Terminologies)

- **Text (T):** The main body of text to search within. Length is denoted as n .
- **Pattern (P):** The substring we want to find. Length is denoted as m .
- **Shift (s):** The alignment offset of the pattern relative to the text. We check if $T[s \dots s + m - 1] = P[0 \dots m - 1]$.

2 Naive / Brute Force String Matching

The **Brute Force** (or Naive) algorithm checks for a pattern match at all possible shift values from $s = 0$ up to $s = n - m$. At each shift, it compares character-by-character from left to right.

Note. Analogy: Imagine trying to open a combination padlock by testing every single numeric combination starting from 000, 001, 002, and so on. If a digit doesn't work, you clear it, move to the next combination, and start testing digits from the first position again.

2.1 Algorithm Mechanics

For each shift s :

1. Align the pattern P at $T[s]$.
2. Compare character $P[j]$ with $T[s + j]$ starting from $j = 0$.
3. If all characters match ($j = m$), report a match at shift s .
4. If a mismatch occurs at any j , increment shift to $s + 1$, reset $j = 0$, and repeat.

2.2 Complexity Analysis

- **Worst-Case Complexity:** $O(m \cdot (n - m + 1))$. If $n \gg m$, this is $O(m \cdot n)$. This occurs when there are many partial matches (e.g., $T = \text{AAAAAAAAAAB}$ and $P = \text{AAAB}$).
- **Best-Case Complexity:** $O(n)$. Occurs when the first character of the pattern mismatch immediately (e.g., $T = \text{BBBBBBBBB}$ and $P = \text{AAA}$).

3 Knuth-Morris-Pratt (KMP) Algorithm

The **Knuth-Morris-Pratt** (KMP) algorithm improves execution speed by never backtracking in the text T . When a mismatch occurs, KMP uses precomputed information about the pattern itself to skip redundant alignments.

Note. Analogy: Imagine reading a text and searching for the word CHARACTER. If you match CHARAC but mismatch at the next letter T, you don't go back in the text to read the second letter H again. You know you've already matched CHARAC, so you can safely shift forward past the matched prefix.

3.1 The Prefix Function (Pi Table / Failure Function)

The core of KMP is the π table of length m . For each index i in the pattern, $\pi[i]$ stores the length of the longest proper prefix of $P[0 \dots i]$ that is also a suffix of $P[0 \dots i]$.

Example 3.1 Computing the π Table

Let's compute the π table for the pattern $P = \text{ABABC}$:

Table 5.1: Prefix Function π Table for $P = \text{ABABC}$

i	0	1	2	3	4
$P[i]$	A	B	A	B	C
$\pi[i]$	0	0	1	2	0

- $\pi[0] = 0$: Substring "A" has no proper prefix.
- $\pi[1] = 0$: Substring "AB" has prefixes {"A"}, suffixes {"B"}. No match.
- $\pi[2] = 1$: Substring "ABA" has prefixes {"A", "AB"}, suffixes {"A", "BA"}. "A" matches (length 1).
- $\pi[3] = 2$: Substring "ABAB" has prefix "AB" which matches suffix "AB" (length 2).
- $\pi[4] = 0$: Substring "ABABC" has no prefix matching suffix because of "C".

3.2 KMP Search Execution

When matching $T[i]$ against $P[j]$:

- If $T[i] == P[j]$, increment both i and j .
- If $j == m$, a match is found. Shift using $j = \pi[j - 1]$.
- If $T[i] \neq P[j]$:
 - If $j \neq 0$, do not decrement i . Instead, update $j = \pi[j - 1]$ and check again.
 - If $j == 0$, increment i by 1.

3.3 Complexity Analysis

- **Preprocessing time:** $O(m)$ to build the π table.
- **Matching time:** $O(n)$ because the index i in the text never decreases.
- **Total Complexity:** $O(n + m)$.

4 Boyer-Moore Algorithm

The **Boyer-Moore** (BM) algorithm is the gold standard for text search in text editors and command-line utilities (like `grep`). It compares characters from **right to left** (starting at the end of the pattern) but shifts the pattern from left to right.

Note. Analogy: Imagine looking for a word in a word search puzzle. Instead of starting at the first letter, you look at the last letter. If that letter isn't even in the word you are looking for, you can jump the entire width of the word forward, saving a massive amount of scanning time.

Boyer-Moore uses two independent heuristics to determine how far it can shift the pattern on a mismatch:

4.1 The Bad Character Heuristic

When a mismatch occurs at $T[i] = c$ and $P[j] \neq c$: 1. If c is not in the pattern P , shift the pattern completely past $T[i]$. 2. If c is in the pattern P , shift the pattern to align the rightmost occurrence of c in P with $T[i]$.

Let $\text{last}[c]$ be the rightmost index of character c in the pattern (0-indexed). If c does not exist in P , $\text{last}[c] = -1$. The shift distance is:

$$\text{Shift} = j - \text{last}[c] \quad (1)$$

4.2 The Good Suffix Heuristic

If a mismatch occurs after matching a suffix t of the pattern: 1. Shift the pattern to align the matched suffix t with its next rightmost occurrence in P . 2. If t does not occur elsewhere, align a prefix of P that matches a suffix of t with T . 3. If no such overlap exists, shift the pattern completely past t .

4.3 Shift Selection

At each mismatch, Boyer-Moore calculates shifts from **both** the Bad Character rule and the Good Suffix rule, and shifts the pattern by the **maximum** of the two values.

- **Worst-Case Complexity:** $O(m \cdot n)$ (occurs with repetitive text).
- **Best-Case Complexity:** $O(n/m)$ (sub-linear speed! We skip the majority of characters).

5 Horspool's Algorithm

Horspool's Algorithm is a simplified version of the Boyer-Moore algorithm. It retains the right-to-left comparison but uses **only** the Bad Character rule. It is highly efficient and simpler to implement.

Instead of looking at the mismatched character in T , Horspool's always determines the shift based on the character in the text aligned with the **last character** of the pattern, regardless of where the mismatch occurred.

5.1 The Shift Table Calculation

Let Σ be the alphabet. For each character $c \in \Sigma$, the shift table value $d(c)$ is:

$$d(c) = \begin{cases} m - 1 - i & \text{where } i \text{ is the rightmost index of } c \text{ in } P[0 \dots m - 2] \\ m & \text{if } c \text{ is not in } P[0 \dots m - 2] \end{cases} \quad (2)$$

Example 5.1 Horspool Shift Table

Let's compute the shift table for pattern $P = \text{BARBER}$ (length $m = 6$):

- Check characters in $P[0 \dots 4] = \text{BARBE}$.
- $d(\text{E}) = 6 - 1 - 4 = 1$
- $d(\text{B}) = 6 - 1 - 3 = 2$ (based on the second B at index 3)
- $d(\text{R}) = 6 - 1 - 2 = 3$
- $d(\text{A}) = 6 - 1 - 1 = 4$
- For any other character (like R at the end, or X), $d(c) = 6$.

5.2 Complexity Analysis

- **Average Case:** $O(n)$
- **Worst Case:** $O(m \cdot n)$

6 Rabin-Karp Algorithm

The **Rabin-Karp** algorithm uses hashing to search for the pattern. It computes a hash value for the pattern and for each window of length m in the text.

Note. Analogy: Imagine trying to find a matching page in a folder of documents. Instead of reading every line on every page, you count the number of words on each page. If a page doesn't have the same number of words as your target, you skip it instantly. If it does match, you do a detailed reading to confirm it is the correct page.

6.1 Rolling Hash Function

To make hashing fast, Rabin-Karp uses a **rolling hash**. The hash of the next window is calculated from the previous window in $O(1)$ time by subtracting the leading character and adding the trailing character.

Let d be the size of the alphabet (radix), and q be a prime number. The hash value of $P[0 \dots m - 1]$ is:

$$p = (d^{m-1}P[0] + d^{m-2}P[1] + \dots + P[m - 1]) \bmod q \quad (3)$$

For a text window $T[s \dots s + m - 1]$ with hash value t_s , the hash of the next window t_{s+1} is:

$$t_{s+1} = (d(t_s - T[s] \cdot h) + T[s + m]) \bmod q \quad (4)$$

where $h = d^{m-1} \bmod q$.

6.2 Handling Collisions

If $p == t_s$, it is a potential match. Because different strings can produce the same hash value (a collision), the algorithm must verify character-by-character:

- If they match exactly, report shift s .
- If not, it is a **spurious hit** (ignore and continue).

6.3 Complexity Analysis

- **Best/Average Case:** $O(n+m)$ (if the hash function is good and causes few collisions).
- **Worst Case:** $O(m \cdot n)$ (occurs if the hash function is bad, causing collisions at every single step, requiring verification every time).

7 Algorithm Comparison Summary

Table 1: Comparison of Pattern Matching Algorithms

Algorithm	Preprocessing	Best Time	Worst Time	Space Complexity	Complexity
Naive / Brute Force	None	$O(n)$	$O(m \cdot n)$	$O(1)$	
KMP	$O(m)$	$O(n)$	$O(n + m)$	$O(m)$	
Boyer-Moore	$O(m + \Sigma)$	$O(n/m)$	$O(m \cdot n)$	$O(m + \Sigma)$	
Horspool	$O(m + \Sigma)$	$O(n/m)$	$O(m \cdot n)$	$O(\Sigma)$	
Rabin-Karp	$O(m)$	$O(n + m)$	$O(m \cdot n)$	$O(1)$	